

helixcode_analysis

HelixCode Repository - Exhaustive Analysis

Analysis Date: 2025-07-01
Repository: <https://github.com/HelixDevelopment/HelixCode>
Branch: main
Purpose: Deep analysis for LLMsVerifier integration as single source of truth for models

1. Repository Structure Overview

1.1 Top-Level Directories

Directory	Purpose
.github/workflows	GitHub Actions workflows (auto-commit style)
Assets	Project logos, wide_black.png, themes
Dependencies	Dependency tracking and governance
Documentation	Technical documentation
Example_Projects	Example projects submodule
Example_Resources	Example resources submodule
Github-Pages-Website	Marketing website submodule (@49f7e16)
HelixCode	Main Go implementation
Implementation_Guide	Implementation plans and guides
Specification	Technical specifications and requirements
Upstreams	Upstream sync scripts (GitLab)
Website	Marketing website

Directory	Purpose
awesome-ai-memory	awesome-ai-memory submodule (@ 7f281fd)
challenges/scripts	Challenge scripts (host power management, no-suspend)
cmd/security_test	Security testing tools
docker	Docker configurations
docs	Additional documentation
internal	Go internal packages (see below)
isolated_files	Isolated/misc files
scripts	Build and utility scripts
security	Security-related files
test	Test configurations
tests/e2e	E2E test suites

1.2 helix_code/ Subdirectory Structure (Main Code)

```

helix_code/
├── cmd/
│   ├── cli/           # CLI client entry point
│   ├── server/        # HTTP server entry point
│   └── security-test/  # Security testing tools
├── internal/
│   ├── agent/         # Agent subsystem
│   ├── auth/          # JWT authentication (VERIFIED REAL)
│   ├── cognee/        # Cognee integration
│   ├── commands/      # Command implementations
│   ├── config/        # Configuration management (Viper-based)
│   ├── context/       # Context management
│   ├── database/      # PostgreSQL layer
│   ├── deployment/    # Deployment tools
│   ├── discovery/     # Discovery services
│   ├── editor/        # Code editing tools
│   ├── event/         # Event system
│   ├── fix/           # Fix utilities
│   ├── focus/         # Focus management
│   ├── hardware/      # Hardware detection
│   ├── hooks/         # Hook system
│   ├── llm/           # LLM providers and reasoning (BLUFF AREA)
│   ├── logging/       # Logging
│   ├── logo/          # Logo processing
│   ├── mcp/           # MCP protocol implementation
│   └── memory/        # Memory integration

```

mocks/	# Test mocks
monitoring/	# Monitoring
notification/	# Multi-channel notifications
performance/	# Performance utilities
persistence/	# Persistence layer
project/	# Project management
provider/	# Provider utilities
providers/	# Provider implementations
redis/	# Redis client
server/	# HTTP server & API (Gin)
session/	# Session management
task/	# Task management
tools/	# Tool ecosystem
user/	# User management
util/	# Utilities
worker/	# SSH-based worker pool
workflow/	# Workflow engine
api/	# API definitions
applications/	# Platform-specific apps (desktop, mobile, Aurora)
assets/	# Generated assets
benchmarks/	# Benchmark suites
bin/	# Build output
config/	# Configuration files
docker/	# Docker files
examples/	# Examples
scripts/	# Build scripts
tests/	# Test suites

Key Finding: The project is primarily **Go (95.4%)**, with some Shell (4.3%), Makefile (0.1%), Swift (0.1%), Kotlin (0.1%), and Dockerfile (0.0%).

2. README & Documentation Analysis

2.1 README.md [2](#)

Project Identity: HelixCode - Distributed AI Development Platform
Package: dev.helix.code
License: MIT
Version: 1.0.0

Documented Features (5 Phases):

Phase	Status	Key Features
Phase 1: Foundation	Completed	PostgreSQL schema (11 tables), JWT auth, worker management, task management, REST API (Gin), config system

Phase	Status	Key Features
Phase 2: Core Services	Completed	Task division, LLM multi-provider, distributed computing, MCP protocol, reasoning, notifications
Phase 3: Workflows	Completed	Project management, dev workflows (plan/build/test/refactor), session management
Phase 4: LLM Integration	Completed	Hardware detection, model management, provider architecture, CLI interface
Phase 5: Advanced Features	Completed	SSH worker pool, LLM tooling, multi-client (REST/CLI/TUI/WebSocket), cross-platform, mobile ready

Documented CLI Usage:

```
# Interactive mode
./cli
# List workers
./cli --list-workers
# Add a worker
./cli --worker worker-host --user helix --key ~/.ssh/id_rsa
# Generate with LLM
./cli --prompt "Hello world" --model llama-3-8b
# Health check
./cli --health
```

Documented API Endpoints: - POST /api/v1/auth/register, POST /api/v1/auth/login, POST /api/v1/auth/refresh - GET /api/v1/workers, POST /api/v1/workers, GET /api/v1/workers/:id - GET /api/v1/tasks, POST /api/v1/tasks, GET /api/v1/tasks/:id, POST /api/v1/tasks/:id/start - GET /api/v1/projects, POST /api/v1/projects, GET /api/v1/projects/:id

Build Commands:

```
make build          # Build application
make test           # Run all tests
make clean          # Clean build artifacts
make lint           # Lint code
make fmt            # Format code
```

3. AGENTS.md Analysis

3.1 AGENTS.md ⁵ — Authoritative Agent Guide

Version: 2.0.0 (CONST-035 Anti-Bluff Mandate)

Date: 2026-04-30

Critical Bluff Areas Identified:

BLUFF-001: LLM Generation is Simulated (CRITICAL)

- **File:** `cmd/cli/main.go` lines 190-214
- **Evidence:** Simulated response with `fmt.Sprintf("Generated response for: %s...")`
- **Fix Priority:** P0 - Immediate

BLUFF-002: Model Listing is Hardcoded (CRITICAL)

- **File:** `cmd/cli/main.go` lines 101-128
- **Evidence:** Only 3 hardcoded models (llama-3-8b, mistral-7b, phi-3-mini)
- **Fix Priority:** P0 - Immediate

BLUFF-003: Command Execution is Simulated (HIGH)

- **File:** `cmd/cli/main.go` lines 237-250
- **Evidence:** `time.Sleep(1 * time.Second)` simulating command execution
- **Fix Priority:** P0 - Immediate

Free AI Providers Listed: - XAI (Grok): grok-3-fast-beta, grok-3-mini-fast-beta - OpenRouter: Free models - GitHub Copilot: gpt-4o, claude-3.5-sonnet (with subscription) - Qwen: 2,000 requests/day free tier

Technology Stack (from AGENTS.md): - Go 1.24.0 with toolchain go1.24.9 - Gin v1.11.0 (HTTP) - JWT v4.5.2 - Viper v1.21.0 (Config) - Cobra v1.8.0 (CLI) - Testify v1.11.1 (Testing) - Fyne v2.7.0 (Desktop UI) - tview v0.42.0 (Terminal UI) - chromedp v0.14.2 (Browser) - goquery v1.10.3 (Web scraping) - go-tree-sitter (Parsing)

4. CONSTITUTION.md Analysis

4.1 Key Constitutional Constraints [34](#)

Constraint	Description
CONST-001	NO CI/CD pipelines - permanent, non-negotiable
CONST-002	NO mocks in production - only in unit tests (*_test.go with -short)
CONST-003	NO HTTPS for Git - SSH only
CONST-004	NO manual container commands - orchestrator-owned
CONST-005	100% real data for non-unit tests
CONST-006	Challenge coverage - every component must have challenge scripts
CONST-017	Zero-Bluff Testing (CONST-035)
CONST-018	Host Power Management Hard Ban - no suspend/hibernate/reboot
CONST-020	Provider Fallback Chain Reality - fallback chains tested with actual failures
CONST-021	No Mocks Above Unit - Makefile must include no-mocks-above-unit target
CONST-025	Secret Management - NO secrets in code, ever
CONST-035	End-User Usability Mandate - every PASS must guarantee quality, completion, usability

5. Configuration System

5.1 Main Config File: `helix_code/internal/config/config.go` [15](#)

Configuration Loading Order (Viper-based): 1. Set defaults (`setDefault()`) 2. Find config file (`findConfigFile()`) 3. Read environment variables (prefix HELIX) 4. Explicitly bind critical env vars: - `HELIX_AUTH_JWT_SECRET` - `HELIX_DATABASE_PASSWORD` - `HELIX_DATABASE_HOST` - `HELIX_REDIS_PASSWORD`

- HELIX_REDIS_HOST - HELIX_REDIS_PORT 5. Unmarshal into Config struct 6.
Validate(validateConfig())

Config Struct (key fields):

```
type Config struct {
    Version      string           `mapstructure:"version"`
    UpdatedBy    string           `mapstructure:"updated_by"`
    Application   ApplicationConfig `mapstructure:"application"`
    Server        ServerConfig      `mapstructure:"server"`
    Database      database.Config    `mapstructure:"database"`
    Redis         RedisConfig       `mapstructure:"redis"`
    Auth          AuthConfig        `mapstructure:"auth"`
    Workers       WorkersConfig     `mapstructure:"workers"`
    Tasks         TasksConfig       `mapstructure:"tasks"`
    LLM           LLMConfig         `mapstructure:"llm"`
    Providers     ProvidersConfig   `mapstructure:"providers"`
    Logging       LoggingConfig     `mapstructure:"logging"`
    Cognee        *CogneeConfig     `mapstructure:"cognee"`
}
```

LLM Configuration:

```
type LLMConfig struct {
    DefaultProvider string           `mapstructure:"default_provider" // default: "local"
    DefaultModel    string           `mapstructure:"default_model"   // default:
    "llama-3.2-3b"
    MaxTokens       int              `mapstructure:"max_tokens"      // default: 4096
    Temperature     float64          `mapstructure:"temperature"     // default: 0.7
}
```

Valid LLM Providers (from validator): local, openai, anthropic, gemini, xai, openrouter, copilot

Default Config Locations: - ./config.yaml - ./config/config.yaml -
\$HOME/.config/helixcode/config.yaml - /etc/helixcode/config.yaml

5.2 Environment Variables (.env.example) [4](#)

```
# Required
HELIX_DATABASE_PASSWORD=CHANGE_ME_db_password
HELIX_AUTH_JWT_SECRET=your-super-secret-jwt-key
HELIX_REDIS_PASSWORD=redispass
```

```
# Ports
HELIX_API_PORT=8080
HELIX_SSH_PORT=2222
HELIX_WEB_PORT=3000

# LLM Provider Keys (optional)
# HELIX_OPENAI_API_KEY=...
# HELIX_OLLAMA_HOST=http://ollama:11434
# HELIX_LLAMA_CPP_HOST=http://llamacpp:8080
```

5.3 ConfigManager

File: helix_code/internal/config/config.go lines 360+

Provides ConfigManager with: - NewHelixConfigManager(configPath) - loads or creates default config - GetConfig(), UpdateConfig(), UpdateConfigFromMap() - ExportConfig(), ImportConfig(), BackupConfig(), ResetToDefaults() - Configuration templates (basic, development, production, testing) - Configuration migrator (1.0.0 -> 1.1.0 -> 1.2.0) - Configuration validator with custom rules

6. Core Architecture

6.1 Entry Points

Entry Point	File	Purpose
Server	cmd/server/main.go 37	HTTP server (Gin) with DB, Redis
CLI	cmd/cli/main.go 10	Command-line interface
CLI (old)	cmd/cli/main.go.old 11	Older CLI with more commands

Server Startup Flow (cmd/server/main.go): 1. Load config (config.Load()) 2. Initialize database (optional - empty host disables) 3. Initialize schema (db.InitializeSchema()) 4. Initialize Redis 5. Create HTTP server (server.New(cfg, db, rds)) 6. Start HTTP server 7. Graceful shutdown on SIGINT/SIGTERM

CLI Startup Flow (cmd/cli/main.go): 1. NewCLI(): - Initialize LLM provider (Ollama, default llama3.2, http://localhost:11434) - Initialize worker pool

(worker.NewSSHWorkerPool(true)) - Initialize notification engine 2. cli.Run() parses flags and dispatches to handlers

6.2 CLI Flag Architecture

```
command          = flag.String("command", "", "Command to execute")
workerHost       = flag.String("worker", "", "Worker host to add")
workerUser       = flag.String("user", "", "Worker SSH username")
workerKey        = flag.String("key", "", "Worker SSH key path")
model            = flag.String("model", "llama-3-8b",
    "LLM model to use")
prompt           = flag.String("prompt", "", "Prompt for LLM
    generation")
maxTokens        = flag.Int("max-tokens", 1000, "Maximum tokens")
temperature      = flag.Float64("temperature", 0.7, "Generation
    temperature")
stream           = flag.Bool("stream", false, "Stream the
    response")
listWorkers      = flag.Bool("list-workers", false, "List all
    workers")
listModels       = flag.Bool("list-models", false, "List available
    models")
healthCheck      = flag.Bool("health", false, "Perform health
    check")
notify           = flag.String("notify", "", "Send notification")
notifyType       = flag.String("notify-type", "info",
    "Notification type")
notifyPriority    = flag.String("notify-priority", "medium",
    "Notification priority")
nonInteractive    = flag.Bool("non-interactive", false, "Run in non-
    interactive mode")
```

Interactive Commands: - workers → handleListWorkers - models → handleListModels - health → handleHealthCheck - help / exit / quit

7. Model/Provider Management System

7.1 Provider Interface [38](#)

File: helix_code/internal/llm/missing_types.go

```
type Provider interface {
    GetType() ProviderType
    GetName() string
    GetModels() []ModelInfo
    GetCapabilities() []ModelCapability
```

```

Generate(ctx context.Context, request *LLMRequest)
    (*LLMResponse, error)
GenerateStream(ctx context.Context, request *LLMRequest, ch
    chan<- LLMResponse) error
IsAvailable(ctx context.Context) bool
GetHealth(ctx context.Context) (*ProviderHealth, error)
Close() error
}

```

7.2 Provider Type Registry [38](#)

35+ Provider Types Defined:

Category	Providers
Cloud APIs	openai, anthropic, gemini, vertexai, azure, bedrock, groq, qwen, copilot, openrouter, xai, cohere, mistral, huggingface
Local Inference	ollama, llamacpp, vllm, localai, fastchat, textgen, lmstudio, jan, koboldai, gpt4all, tabbyapi, mlx, mistralrs
Memory/Agent	memgpt, crewai, characterai, replika, anima, llamaindex
Database/Vector	clickhouse, supabase, deeplake, chroma
Generic	local, agnostic, gemma

7.3 Provider Factory [21](#)

File: helix_code/internal/llm/factory.go

```

func NewProvider(config ProviderConfigEntry) (Provider, error) {
    switch config.Type {
    case ProviderTypeOpenAI:      return
        NewOpenAIProvider(config)
    case ProviderTypeAnthropic:   return
        NewAnthropicProvider(config)
    case ProviderTypeGemini:      return
        NewGeminiProvider(config)
    case ProviderTypeOllama:      return
        NewOllamaProvider(ollamaConfig)
    case ProviderTypeLlamaCpp:    return
        NewLlamaCPPProvider(llamaConfig)
    }
}

```

```

    case ProviderTypeQwen:          return NewQwenProvider(config)
    case ProviderTypeXAI:           return NewXAIProvider(config)
    case ProviderTypeOpenRouter:    return
        NewOpenRouterProvider(config)
    case ProviderTypeCopilot:       return
        NewCopilotProvider(config)
    case ProviderTypeAzure:         return NewAzureProvider(config)
    case ProviderTypeBedrock:       return
        NewBedrockProvider(config)
    case ProviderTypeVertexAI:      return
        NewVertexAIProvider(config)
    case ProviderTypeGroq:          return NewGroqProvider(config)
    case ProviderTypeVLLM:          return NewVLLMProvider(config)
    case ProviderTypeLocalAI:       return
        NewLocalAIProvider(config)
    // ... 20+ more cases
    default:
        return nil, fmt.Errorf("unsupported provider type: %s",
            config.Type)
}

```

Model Manager Initialization:

```

func InitializeModelManager(configs []ProviderConfigEntry)
    (*ModelManager, error) {
    manager := NewModelManager()
    for _, config := range configs {
        if !config.Enabled { continue }
        provider, err := NewProvider(config)
        if err != nil { return nil, err }
        if err := manager.RegisterProvider(provider); err != nil
        {
            return nil, err
        }
    }
    return manager, nil
}

```

7.4 Model Manager [33](#)

File: helix_code/internal/llm/model_manager.go

Core Responsibilities: - RegisterProvider(provider Provider) error - registers provider and its models - SelectOptimalModel(criteria ModelSelectionCriteria) (*ModelInfo, error) - selects best model - GetAvailableModels() []*ModelInfo - returns all registered models - GetModelsByCapability(capabilities []ModelCapability)

[]*ModelInfo - capability filtering - GetProviderForModel(modelName, providerType) (Provider, error) - provider lookup - HealthCheck(ctx) map[ProviderType]*ProviderHealth - health checks all providers

Model Scoring Criteria:

```
type ModelSelectionCriteria struct {
    TaskType                string
    RequiredCapabilities    []ModelCapability
    MaxTokens               int
    Budget                  float64
    LatencyRequirement      time.Duration
    QualityPreference       string // "fast", "balanced", "quality"
}
```

Scoring Algorithm: 1. Capability matching (must have all required) 2. Context size adequacy 3. Task type suitability (bonus multipliers for code_generation, debugging, etc.) 4. Hardware compatibility (hardware.Detector.CanRunModel()) 5. Quality preference (70B = 1.3x, 3B = 0.9x)

7.5 Model Discovery Engine [29](#)

File: helix_code/internal/llm/model_discovery.go

Key Features: - GetRecommendations(ctx, RecommendationRequest) (*RecommendationResponse, error) - Hardware-aware model recommendations - Task compatibility scoring (code_generation, planning, debugging, testing, refactoring, documentation, analysis) - Performance estimation (tokens/second, memory, latency, cost) - Privacy scoring (local/hybrid/cloud) - Fallback alternatives based on model category

Hardcoded External Models (in fetchExternalModels):

```
[ ]*ModelInfo{
    {ID: "llama-3-8b-instruct", Name: "Llama 3 8B Instruct",
      Format: FormatGGUF, Size: 4.7GB, ContextSize: 8192},
    {ID: "mistral-7b-instruct", Name: "Mistral 7B Instruct",
      Format: FormatGGUF, Size: 4.1GB, ContextSize: 32768},
    {ID: "codellama-7b-instruct", Name: "CodeLlama 7B Instruct",
      Format: FormatGGUF, Size: 3.8GB, ContextSize: 16384},
}
```

7.6 Cross-Provider Registry [14](#)

File: helix_code/internal/llm/cross_provider_registry.go

Default Providers Registered:

Provider	Type	Endpoint	Default Port	Supported Formats
vllm	openai-compatible	http://localhost:8000	8000	GGUF, GPTQ, AWQ, HF, FP16, BF16
ollama	llamacpp	custom http://localhost:8080	8080	GGUF
ollama	openai-compatible	http://localhost:11434	11434	GGUF

Registry persisted to ~/.helixcode/local-llm/registry.json

7.7 Alias Management [20](#)

File: helix_code/internal/llm/aliases.go

- AliasManager with fuzzy matching (Levenshtein distance)
- Default threshold: 0.7 (70%)
- Provider-specific alias resolution
- Autocomplete support

7.8 Auto LLM Manager [23](#)

File: helix_code/internal/llm/auto_llm_manager.go

Zero-Touch Mode: - Auto-discover, auto-install, auto-configure, auto-start, auto-monitor, auto-update - Background tasks: health monitor, performance optimizer, update checker - Auto-recovery with retry limits - Git-based provider installation (clone, build, configure) - Directory structure under ~/.helixcode/local-llm/

7.9 Ollama Provider Implementation [31](#)

File: helix_code/internal/llm/ollama_provider.go

Real Implementation (anti-bluff verified): - discoverModels() - calls GET /api/tags to fetch actual models - Generate() - calls POST /api/chat with real HTTP request - GenerateStream() - calls streaming endpoint, sends real response chunks - IsAvailable() - tests API endpoint - GetHealth() - tests API endpoint with latency measurement

Configuration:

```
type OllamaConfig struct {
    BaseURL      string           // default: http://localhost:11434
    DefaultModel string           // e.g., "llama2"
    Timeout      time.Duration   // default: 120s
    KeepAlive    time.Duration
    StreamEnabled bool
}
```

7.10 Current Model Listing (CRITICAL BLUFF)

File: helix_code/cmd/cli/main.go lines 101-128

```
func (c *CLI) handleListModels(ctx context.Context) error {
    models := []struct {
        ID          string
        Name         string
        Provider     string
        ContextSize  int
        Status       string
    }{
        {"llama-3-8b", "Llama 3 8B", "llama.cpp", 8192,
         "available"},
        {"mistral-7b", "Mistral 7B", "ollama", 4096,
         "available"},
        {"phi-3-mini", "Phi-3 Mini", "openai", 128000,
         "available"},
    }
    // ... prints hardcoded list
}
```

This is HARDCODED - NOT dynamic discovery. The AGENTS.md explicitly flags this as BLUFF-002 (P0 critical).

8. CLI UX Analysis

8.1 Current CLI (cmd/cli/main.go) [10](#)

UI Framework: Standard Go flag package (NOT Cobra - despite AGENTS.md claiming Cobra)

Commands: | Flag | Handler | |——|——| | --list-workers |
handleListWorkers | | --list-models | handleListModels (HARDCODED) |
| --health | handleHealthCheck | | --worker HOST | handleAddWorker | |
--prompt TEXT | handleGenerate (REAL - calls provider.Generate) | | --model
NAME | Used with --prompt | | --stream | Used with --prompt | | --notify
MESSAGE | handleNotification | | --command CMD | handleCommand
(SIMULATED) | | (no flags) | handleInteractive |

Interactive Mode: - Prompt: helix> - Commands: workers, models, health, help, exit/quit - Uses fmt.Scanln() for input - Signal handling for graceful shutdown (SIGINT/SIGTERM)

8.2 Older CLI (cmd/cli/main.go.old) [11](#)

UI Framework: Argument-based (not flags)

Commands: - help, version, models, hardware, health - chat <prompt>, generate <description>, plan <project>, test <code>, debug <issue>, refactor <code>

Status: All AI commands are TODO placeholders:

```
func (c *CLI) startChat(args []string) error {  
    // TODO: Implement actual chat functionality  
    fmt.Println("Chat functionality coming soon...")  
    return nil  
}
```

9. Platform Support

Documented Platforms: - Linux (primary) - macOS (Intel + ARM64) - Windows - Aurora OS - SymphonyOS / Harmony OS - Android (mobile framework) - iOS (mobile framework)

Build Targets (from Makefile):

```
make prod                # Linux, macOS, Windows binaries  
make desktop-linux       # Desktop GUI + CLI for Linux  
make desktop-macos      # Intel + ARM  
make desktop-windows     # .exe  
make mobile-ios          # HelixCore.xcframework  
make mobile-android      # mobile.aar  
make aurora-os           # Aurora OS client  
make harmony-os          # Harmony OS client
```

10. Test Framework

10.1 Testing Infrastructure

Framework: Go built-in testing + Testify v1.11.1

Test Categories (from AGENTS.md): 1. Unit tests - Mocks OK, *_test.go, -short flag 2. Contract tests - Real API schemas, no mocks 3. Component tests - Real subsystems wired together 4. Integration tests - Full app with real dependencies 5. E2E challenges - Complete user workflows 6. Security tests - OWASP compliance 7. Performance tests - Benchmarks

Makefile Test Targets ²⁴:

```
make test          # Basic go test -v ./...
make test-all     # test + coverage + benchmark + docs
make test-coverage # go test -v -race
                  -coverprofile=coverage.out
make test-benchmark # go test -bench=. -benchmem ./...
make test-docs     # Documentation validation
make test-full     # ALL tests with infrastructure (ZERO
                  skips)
make test-unit-full # Unit tests with full infra
make test-integration-full # Integration tests
make test-e2e-full  # E2E challenge tests
make test-security-full # Security tests
make test-load-full # Load tests
make test-complete  # All test types in sequence
make coverage-full  # Comprehensive coverage report
```

Test Infrastructure:

```
make test-infra-up    # docker compose -f docker-compose.full-
                    test.yml up -d
make test-infra-down  # Stop infrastructure
make test-infra-status # Check status
```

10.2 LLM Package Test Files

File: helix_code/internal/llm/ contains extensive test files: -

- aliases_test.go
- anthropic_provider_test.go
- auto_llm_manager_test.go
- azure_provider_test.go
- bedrock_provider_test.go
- cache_control_test.go
- cloud_providers_integration_test.go
- copilot_provider_test.go
- cross_provider_registry_test.go
- cross_provider_test.go
- factory_test.go
- local_providers_integration_test.go
- local_providers_test.go
- model_converter_test.go
- model_discovery_test.go
- model_download_manager_test.go
- model_manager_test.go
- ollama_provider_test.go
- openai_compatible_provider_test.go
- openai_provider_test.go
- openrouter_provider_test.go
- provider_features_test.go
- provider_registry_test.go
- qwen_oauth_test.go
- qwen_provider_test.go
- reasoning_test.go
- token_budget_test.go
- tool_provider_test.go
- usage_analytics_test.go
- vertexai_provider_test.go
- xai_provider_test.go

10.3 Challenge System

Location: challenges/scripts/ [22](#)

Files: - host_no_auto_suspend_challenge.sh - CONST-033 host power management guard - no_suspend_calls_challenge.sh - No suspend calls challenge

E2E Tests: tests/e2e/challenges/ with run_all_challenges.sh

Test Runner: cd tests/e2e/challenges && go run cmd/runner/main.go -all

11. Dependencies Analysis

11.1 go.mod [3](#)

Module: dev.helix.code

Go Version: 1.24.0 (toolchain go1.24.9)

Key Dependencies:

Dependency	Version	Purpose
github.com/gin-gonic/gin	v1.11.0	HTTP web framework
github.com/spf13/cobra	v1.8.0	CLI framework (claimed but NOT used in current CLI)
github.com/spf13/viper	v1.21.0	Configuration management
github.com/golang-jwt/jwt/v4	v4.5.2	JWT authentication
github.com/jackc/pgx/v5	v5.7.6	PostgreSQL driver
github.com/redis/go-redis/v9	v9.17.2	Redis client
github.com/stretchr/testify	v1.11.1	Testing framework
fyne.io/fyne/v2	v2.7.0	Desktop GUI
github.com/rivo/tview	v0.42.0	Terminal UI
	v0.14.2	Browser automation

Dependency	Version	Purpose
github.com/chromedp/chromedp		
github.com/PuerkitoBio/goquery	v1.10.3	Web scraping
github.com/smacker/go-tree-sitter	-	Parsing
github.com/gorilla/websocket	v1.5.3	WebSocket
github.com/google/uuid	v1.6.0	UUID generation
github.com/fatih/color	v1.18.0	Terminal colors
golang.org/x/crypto	v0.46.0	Cryptography
github.com/hashicorp/golang-lru/v2	v2.0.7	LRU cache

Cloud Provider SDKs: | Dependency | Version | Purpose | |—————|—————|
 ————| | github.com/Azure/azure-sdk-for-go/sdk/azcore | v1.16.0 |
 Azure core | | github.com/Azure/azure-sdk-for-go/sdk/azidentity |
 v1.8.0 | Azure identity | | github.com/aws/aws-sdk-go-v2 | v1.32.7 | AWS
 SDK | | github.com/aws/aws-sdk-go-v2/service/bedrockruntime | v1.23.1
 | AWS Bedrock | | google.golang.org/grpc | v1.76.0 | gRPC | | golang.org/
 x/oauth2 | v0.32.0 | OAuth2 |

Memory Integrations: | Dependency | Version | Purpose | |—————|—————|
 ————| | github.com/getzep/zep-go/v3 | v3.10.0 | Zep memory |

12. Integration Points

12.1 LLM Provider Integration

Current Integration Model: - Provider interface at `helix_code/internal/llm/missing_types.go` - Factory pattern at `helix_code/internal/llm/factory.go` - Each provider is a separate Go file implementing the Provider interface - 35+ provider types supported

Provider Registration Flow: 1. Config specifies `ProviderConfigEntry` (type, endpoint, API key, models, enabled, parameters) 2. `InitializeModelManager(configs)` iterates enabled configs 3. `NewProvider(config)` dispatches to provider constructor 4. `manager.RegisterProvider(provider)` adds to registry

12.2 Database Integration

- PostgreSQL 15+ (optional - empty host disables)
- 11 tables for distributed computing
- Schema initialization in internal/database
- Redis 7+ for caching (optional)

12.3 Worker Pool Integration

- SSH-based worker pool
- Health monitoring
- Auto-installation capability
- Cross-platform workers

12.4 MCP Protocol

- Model Context Protocol implementation
- Multi-transport support

12.5 Notification System

- Multi-channel: Slack, Discord, Email, Telegram
- Notification engine with priority levels

13. Docker Architecture

13.1 docker-compose.yml [36](#)

Services: | Service | Image | Ports | Purpose | |———|———|———|———| |
helixcode-server | Build from Dockerfile | 8080, 2222 | Main application | |
postgres | postgres:15 | 5432 | Database | | redis | redis:7-alpine | 6379 | Cache |
| nginx | nginx:alpine | 80, 443 | Reverse proxy | | prometheus | prom/
prometheus:latest | 9090 | Monitoring | | grafana | grafana/grafana:latest | 3000
| Dashboards |

Health Checks: - Server: curl -f http://localhost:8080/health -
PostgreSQL: pg_isready -U helix - Redis: redis-cli --raw incr ping

14. Gaps, TODOs, and Incomplete Features

14.1 Critical Gaps (P0)

1. **BLUFF-001: LLM Generation Simulated** (file: `cmd/cli/main.go.old`)
 - Current `cmd/cli/main.go` has REAL implementation calling `provider.Generate()`
 - Status: **PARTIALLY FIXED** in current `main.go`, but old file still exists with simulated code
2. **BLUFF-002: Model Listing Hardcoded**
 - `cmd/cli/main.go` lines 101-128: Only 3 hardcoded models
 - Does NOT use `modelManager.GetAvailableModels()` or `provider.GetModels()`
 - Status: **NOT FIXED** - P0 critical
3. **BLUFF-003: Command Execution Simulated**
 - `cmd/cli/main.go` lines 237-250: `time.Sleep(1 * time.Second)` simulating execution
 - Status: **NOT FIXED** - P0 critical
4. **CLI Uses `flag` NOT Cobra**
 - `AGENTS.md` claims Cobra v1.8.0 is the CLI framework
 - Actual `cmd/cli/main.go` uses Go standard `flag` package
 - `cmd/cli/main.go.old` also does NOT use Cobra
 - Status: **DISCREPANCY** - either docs are wrong or Cobra integration is missing
5. **Model Discovery Not Connected to CLI**
 - `ModelDiscoveryEngine` exists with sophisticated scoring
 - CLI `handleListModels` does NOT use it
 - `handleGenerate` does NOT use `SelectOptimalModel()`
 - Status: **NOT INTEGRATED**
6. **No Dynamic Model Source**
 - Models are hardcoded in `model_discovery.go` (`fetchExternalModels`)
 - No external API integration for fetching current model lists
 - No integration with model registries (HuggingFace, Ollama library, etc.)
 - Status: **HARDCODED MODELS ONLY**

14.2 Medium Gaps

1. **Old CLI File Still Present**
 - `cmd/cli/main.go.old` contains simulated code and TODOs
 - Should be removed or archived
2. **Missing Provider Implementations**
 - 35+ provider types defined in `missing_types.go`
 - Only ~15 have actual implementation files visible

- Some may be stubs

3. Hardware Detection Integration

- `hardware.Detector` exists
- `ModelManager.calculateHardwareCompatibility()` calls it
- But CLI does NOT use hardware-aware model selection

4. No LLMsVerifier Integration

- No references to “LLMsVerifier”, “verifier”, or external model verification service
- No API client for fetching verified model lists
- Status: **NOT PRESENT**

15. Current Model Management Approach Summary

15.1 How Models Are Currently Managed

1. **ProviderConfigEntry** structs define which providers are enabled and which models they serve
2. **Factory** creates provider instances based on type
3. **ModelManager** registers providers and maintains a `modelRegistry` map keyed by `providerType:modelName`
4. **ModelDiscoveryEngine** provides recommendations but with hardcoded external models
5. **CrossProviderRegistry** tracks downloaded models and provider compatibility
6. **AliasManager** handles fuzzy model name resolution

15.2 Model Sources

Source	Status	Location
Provider discovery (Ollama /api/tags)	REAL	<code>ollama_provider.go:discoverModels()</code>
CrossProviderRegistry downloaded models	PERSISTED	<code>~/.helixcode/local-llm/registry.json</code>
Hardcoded external models	HARDCODED	<code>model_discovery.go:fetchExternalModels()</code>
CLI list models	HARDCODED	<code>cmd/cli/main.go:handleListModels()</code>

15.3 API Keys / Credentials

- Stored in environment variables (prefix `HELIX_`)

- Configured via `ProviderConfigEntry.APIKey`
 - No secret management beyond env vars (no Vault integration visible)
-

16. Recommendations for LLMsVerifier Integration

16.1 Integration Points

To integrate LLMsVerifier as the single source of truth for models, the following integration points should be targeted:

1. **`internal/llm/model_discovery.go:fetchExternalModels()`**
 - Replace hardcoded model list with LLMsVerifier API call
 - Add LLMsVerifier client to fetch verified models
2. **`internal/llm/model_manager.go:SelectOptimalModel()`**
 - Augment with LLMsVerifier verification status
 - Filter out unverified models or warn about them
3. **`cmd/cli/main.go:handleListModels()`**
 - Replace hardcoded list with dynamic fetch from LLMsVerifier
 - Show verification status for each model
4. **`internal/llm/factory.go:NewProvider()`**
 - Add LLMsVerifier-aware provider initialization
 - Validate provider endpoints against LLMsVerifier registry
5. **New Package: `internal/llm/verifier/`**
 - Create dedicated LLMsVerifier client package
 - Handle API authentication, caching, error handling

16.2 Files to Modify

File	Lines	Change
<code>cmd/cli/main.go</code>	101-128	Replace hardcoded models with LLMsVerifier fetch
<code>internal/llm/model_discovery.go</code>	~900+	Replace <code>fetchExternalModels()</code> hardcoded list
<code>internal/llm/model_manager.go</code>	~280	Add LLMsVerifier status to model scoring
<code>internal/config/config.go</code>	~100	Add LLMsVerifier configuration section
<code>helix_code/go.mod</code>	-	Add LLMsVerifier client dependency

17. Citations

Analysis compiled from exhaustive repository exploration. All file paths are relative to repository root unless otherwise specified.